



Taller 1

QuickSort con Dos Pivotes

Para este taller el objetivo será implementar una variante del algoritmo de ordenamiento QuickSort. Como se vió en clases este algoritmo utilizar un elemento pivote para ordenar, la versión que tendrás que implementar en este taller usará **2 pivotes**.

a) Sección Guiada: TwoPartition

El primer objetivo es implementar la función **TwoPartition** que utiliza dos pivotes, a los que llamaremos **pivot1** y **pivot2**, para separar los elementos del arreglo a ordenar en tres arreglos auxiliares:

- **left**: elementos menores que **pivot1**.
- **middle**: elementos mayores o iguales que **pivot1** y menores o iguales que **pivot2**.
- **right**: elementos mayores que **pivot2**.

Para esto deberás implementar, la siguiente función en C:

```
1 void TwoPartition(  
2     int *array,  
3     int size,  
4     int *pivot1,  
5     int *pivot2,  
6     int *left,  
7     int *left_size,  
8     int *middle,  
9     int *middle_size,  
10    int *right,  
11    int *right_size  
12 )
```

El siguiente pseudocódigo describe la implementación de esta función:

TwoPartition

```
1: pivot1 ← el menor de los siguientes valores: array[0] y array[size - 1]
2: pivot2 ← el mayor de los siguientes valores: array[0] y array[size - 1]
3: left_size ← 0
4: middle_size ← 0
5: right_size ← 0
6: for i ← 1 hasta size - 2 do
7:   if array[i] < pivot1 then
8:     Agregar array[i] a left
9:     Incrementar left_size
10:  else if array[i] > pivot2 then
11:    Agregar array[i] a right
12:    Incrementar right_size
13:  else
14:    Agregar array[i] a middle
15:    Incrementar middle_size
16:  end if
17: end for
```

Nota: Ten en cuenta que las variables `pivot1`, `pivot2`, `left_size`, `middle_size` y `right_size` que recibe la función son punteros, por lo que debes trabajar con ellos como tal.

b) Sección Autónoma: QuickSort con Dos Pivotes

En esta sección deberás utilizar la función `TwoPartition` implementada anteriormente para construir una variante de *QuickSort* que utiliza dos pivotes. **Importante:** En caso de que no hayas completado la función `TwoPartition`, tendrás disponible una versión alternativa `AltTwoPartition` en la librería `libbedd`, la cual escoge los pivotes de forma distinta, pero cumple el mismo objetivo.

Deberás implementar las siguientes dos funciones:

Función rebuild

Implementa la función:

```
1 void rebuild(
2     int *array,
3     int *left,
4     int left_size,
5     int pivot1,
6     int *middle,
7     int middle_size,
8     int pivot2,
9     int *right,
10    int right_size
11 )
```

Esta función deberá reconstruir `array` utilizando los elementos recibidos en el siguiente orden:

`left pivot1 middle pivot2 right`

Los arreglos `left`, `middle` y `right` ya estarán ordenados al momento de llamar a esta función.

Por ejemplo, si `left = [1, 2]`, `pivot1 = 3`, `middle = [4, 5, 6]`, `pivot2 = 7` y `right = [8, 9]`, al finalizar la función, `array` deberá contener:

`[1, 2, 3, 4, 5, 6, 7, 8, 9]`

Función `quicksort`

Implementa la función:

```
1 | void quicksort(int *array, int size)
```

La función deberá ordenar `array` utilizando las funciones `TwoPartition` y `rebuild` implementadas anteriormente.

Lo primero a realizar es inicializar (usando `malloc` o `calloc`) los arreglos auxiliares que usará `partition`: `left`, `middle` y `right`. Además también se deben declarar las variables `pivot1`, `pivot2`, `left_size`, `middle_size` y `right_size`. Después de inicializar y declarar, se ejecuta el llamado a la función `TwoPartition`, recuerda que esta recibe punteros a las variables `pivot1`, `pivot2`, `left_size`, `middle_size` y `right_size`.

Finalmente se hacen llamados recursivos a `quicksort` para ordenar las partes auxiliares, terminando el algoritmo al reconstruir del arreglo original usando la función `rebuild`.

Input

Primero recibirás un entero **E** que indica la cantidad de eventos a procesar. Luego de ello, recibirás **E** eventos, donde cada uno representa un llamado a una de las funciones que debes implementar.

El formato de llamada de cada función es el siguiente:

- **TWOPARTITION**: Se recibe el nombre del evento seguido de un entero que corresponde al tamaño del arreglo sobre el cual se ejecutará el algoritmo. En la línea siguiente se reciben los elementos de este arreglo.
- **REBUILD**: Primero se recibe el nombre del evento, los siguientes parametros se reciben en el siguiente orden y separados por una línea:
 1. El tamaño del arreglo **left** seguido de los elementos que contiene.
 2. El elemento que corresponde al pivote 1.
 3. El tamaño del arreglo **middle** seguido de los elementos que contiene.
 4. El elemento que corresponde al pivote 1.
 5. El tamaño del arreglo **right** seguido de los elementos que contiene.
- **TWOQUICKSORT**: Se recibe el nombre del evento seguido de un entero que corresponde al tamaño del arreglo sobre el cual se ejecutará el algoritmo. En la línea siguiente se reciben los elementos de este arreglo.

Como ejemplo tenemos el siguiente input:

input	
1	4
2	TWOPARTITION 6
3	2 6 4 3 1 5
4	TWOPARTITION 3
5	1 2 3
6	REBUILD
7	3 2 4 6
8	7
9	1 9
10	10
11	1 12
12	TWOQUICKSORT 8
13	6 2 3 9 4 7 10 11

Output

Output esperado del ejemplo de input anterior:

output

```
1 Resultado de la ejecucion de TwoPartition:
2 Pivot1: 2
3 Pivot2: 5
4 Left: 1
5 Middle: 4 3
6 Right: 6
7 Resultado de la ejecucion de TwoPartition:
8 Pivot1: 1
9 Pivot2: 3
10 Left:
11 Middle: 2
12 Right:
13 Resultado de la ejecucion de Rebuild:
14 2 4 6 7 9 10 12
15 Arreglo ordenado con TwoQuickSort:
16 2 3 4 6 7 9
17 10 11
```

Entorno de trabajo

En su carpeta de trabajo tendrán al menos los siguientes elementos:

- `src/`: carpeta con su código
- `Makefile`: archivo para compilar
- `tests/`: carpetas de tests públicos

Comandos (Terminal)

Para compilar:

```
make
```

Para ejecutar un test:

```
./PROGRAMA RUTA_ARCHIVO_INPUT RUTA_ARCHIVO_OUTPUT
```

Donde `PROGRAMA` es el nombre del archivo ejecutable que se genera al compilar, el cual puedes encontrar en la raíz de la carpeta de trabajo. Ejemplos de ejecutables: `troops`.

Además, `RUTA_ARCHIVO_INPUT` es la ruta al archivo de input del test y `RUTA_ARCHIVO_OUTPUT` es la ruta al archivo de output generado por el programa para el test. Ejemplo de `RUTA_ARCHIVO_INPUT` y `RUTA_ARCHIVO_OUTPUT`: `tests/easy-0/input.txt` y `output.txt`.

Importante: Para `RUTA_ARCHIVO_OUTPUT` no uses el archivo de output que viene dentro de un test. El parámetro `RUTA_ARCHIVO_OUTPUT` indica dónde tu programa escribirá el output que genere, por lo que usar una ruta como por ejemplo: `tests/test-1/output.txt` lo que producirá es que tu programa sobrescriba el output esperado por el test.

Para debuggear errores y/o leaks de memoria usando **valgrind**:

```
valgrind ./PROGRAMA RUTA_ARCHIVO_INPUT RUTA_ARCHIVO_OUTPUT
```

Para ejecutar todos los tests:

```
make test
```

Proceso de Entrega

Conexión a internet

Una vez se inicie la imagen y aparezca el escritorio, selecciona el ícono de redes abajo a la derecha. Luego: *"Available Networks" → "eduroam"*. Para conectarse a eduroam, deben tener la siguiente configuración:

- En la opción de *"Authentication"* seleccionar *"Tunneled TLS"*.
- Tener seleccionada la opción *"No CA certificate is required"*.
- En la opción de *"Inner authentication"* seleccionar *"PAP"*
- Luego debe ingresar tus credenciales usuario UC. **IMPORTANTE:** en el usuario debes incluir el @, no sólo el nombre. Ejemplos correctos: *"nombre@uc.cl"*, *"nombre@estudiante.uc.cl"*. Incorrecto: 'nombre' sin el @.

Descargar taller

Para descargar el código base que con el que trabajarán, deben ingresar al Canvas del curso usando el navegador (Firefox). Aquí encontrarán un archivo `.zip` con el código base del taller. Puedes descomprimir el archivo **usando el explorador de archivos**.

Entregar Taller

Para entregar, simplemente deben comprimir su carpeta de trabajo en un archivo **.zip**, y subir este archivo comprimido al buzón de Canvas que tendrán disponible para el taller. Para comprimir su carpeta de trabajo pueden hacerlo **desde el explorador de archivos**.

Evaluación

La nota de tu taller es calculada a partir de test cases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán test cases públicos para que puedas probar tu implementación durante el taller.

Integridad académica e IA

Cualquier acto deshonesto o fraude académico está estrictamente prohibido. Se prohíbe el uso de herramientas de inteligencia artificial y el uso o manipulación de otros dispositivos electrónicos como laptops, tablets, y/o teléfonos durante la evaluación.